CS6290 HPCA — Midterm Notes

Lessons: Intro → Metrics → Pipelining → Branches → Predication → ILP → Tomasulo → ROB → Memory Ordering → Compiler ILP → VLIW → Cache Review → Virtual Memory → Advanced Caches (compiled from the lecture transcripts)

0 · Formula Sheet

Topic	Formula / takeaway
Dynamic power	$P_{dyn} = ½ \cdot C \cdot V^2 \cdot f \cdot \alpha$ (C ∝ area, α = activity factor). Lower V → static (leakage) power ↑; total power has an optimal V.
Chip cost	`cost = wafer cost / #good chips`; yield = good/total; bigger chip → cost grows **super-linearly** (fewer chips + lower yield)
Speedup	$S = Lat_{old}/Lat_{new} = Thr_{new}/Thr_{old}$; S < 1 = slowdown
Iron Law	`CPU time = IC × CPI × T_clk`; mixed CPIs: $\Sigma(IC_i \times CPI_i) \times T_{clk}$; $T_{clk} = 1/f$
Amdahl	$S = 1 / [(1-F) + F/S_e]$, **F = fraction of execution TIME** (not of instructions!)
Lhadma	Don't make the uncommon case too slow: `1/[0.1/0.1 + 0.9/2] = 0.69` (slowdown)
Averaging	Average the times then take the ratio, or use the **geometric mean** for both times and speedups; **never** arithmetic-mean the speedups
Pipeline	n instructions, k stages: `k + (n−1)` cycles; `CPI = 1 + Σ(event freq × cost)`
Branch CPI	$CPI = CPI_{ideal} + (mispred/inst) \times penalty$; mispred/inst = (mispred/branch)×(branch/inst); resolved in stage k → penalty = k−1
N-bit history	Cost per entry $N + 2 \cdot 2^N$ bits; predicts every repeating pattern of length **≤ N+1**
If-conversion	If-converted = both paths + select instructions; branchy = avg path + mispred × penalty; set equal → break-even accuracy
ILP vs IPC	ILP = IPC of an ideal processor (RAW only); **IPC ≤ ILP**
AMAT	`AMAT = HT + MR × MP`; miss time = HT + MP; multi-level: $MP_{L1} = HT_{L2} + MR_{L2} \times MP_{L2}$ (MR_{L2} = **local**)
Cache address	`offset = log₂(block)`, `#sets = size/(block×ways)`, `index = log₂(#sets)`, tag = rest
Replacement bits	LRU: N counters of $\log_2 N$ bits per set; NMRU: $\log_2 N$ bits per set; PLRU: 1 bit per line
Flat PT	`#PTE = VA space / page size`; `size = #PTE × PTE size` (independent of memory actually used and of physical memory size)
Multi-level PT	size = (# tables actually needed at each level) × (each table = 2^{bits} × PTE size)
VIPT, no aliasing	`cache size ≤ associativity × page size` (index + offset bits all inside the page offset)
Prefetch distance	`pdist ≈ memory latency / cycles per iteration`; check the prefetched data is not evicted before use

1 · Introduction

- Computer architecture = designing a computer well suited for its **intended use** (performance, battery life, size, weight, energy efficiency, cost, new abilities like 3D graphics/debugging/security). Building faster transistors is **not** architecture.
- Must anticipate technology trends: designing with current tech yields an obsolete machine. **Moore's Law**: 2× transistors on the same area every 18–24 months → architects aim for 2× speed, ½ energy per op, 2× memory capacity.
- **Memory wall**: processor speed and memory capacity ~2×/2 yrs, but memory **latency** only ~1.1×/2 yrs → gap keeps growing → caches are the "stairs".
- Speed/cost/power trade-off: the same tech step can give "2× speed, same cost, same power" or "1.1× speed, ½ cost, ½ power". Extreme designs (the "Burnium") are usually worst.

Power

- Dynamic (active): $½CV^2f\alpha$. Shrink + 2 cores → C unchanged; lowering voltage is key (V^2). E.g. V×0.8 → 0.64×.
- Static: mostly leakage; grows as V drops → can't lower V indefinitely; total power vs V curve has a minimum.

Ex. Given 30 W at 1 V / 2 GHz. 0.9 V / 1.8 GHz: $30 \times 0.9^2 \times 0.9$ = **21.9 W**; 1.5 V / 3 GHz: $30 \times 1.5^2 \times 1.5$ = **101.3 W**.

Fabrication cost

Ex. Wafer $5000, 10 defects: small chip 400/wafer → 390 good = **$12.80**; 4× area 96/wafer → 86 good = **$58.20** (~5×, not 4×); another 4× area 20/wafer, defects may share a chip → lose 9 → 11 good = **$454** (~8×).

Two ways to use Moore's Law: (1) same functionality, smaller and much cheaper (new devices: phones, watches); (2) same area, more capable (high performance). 1 mm^2 vs 100 mm^2 chip: cost is **>100×**.

2 · Metrics & Evaluation

- **Latency** (start → done) vs **Throughput** (things done per second); throughput ≠ 1/latency (pipelining, replicated servers). E.g. 2 servers, 1 ms/order each → latency 1 ms, throughput 2000/s.
- "X is N times faster than Y": $N = Lat_Y/Lat_X$. 4 h → 10 min: speedup = 240/10 = **24**; the reverse = 0.04.
- Benchmark types: real applications (most representative, hardest to set up) → application **kernels** (real code, but only part of it: the time-consuming loop) → **synthetic** (behaves like a kernel, not real code; good for early design studies) → **peak performance** (marketing only).
- Standards: **TPC** (databases/web/transaction processing), **EEMBC** (embedded: cars, printers, phones), **SPEC** (workstations/CPUs, little I/O; gcc, bwaves, lbm, perl, bzip…).

Ex. X: 9, 10, 5 s; Y: 18, 7, 11 s. Avg times 8 vs 12 → speedup 1.5; arithmetic mean of speedups 1.63 (**wrong**); geometric means 7.66, 11.15 → 1.456 = geometric mean of speedups.

Iron Law / Amdahl

- IC ← algorithm, compiler, ISA; CPI ← ISA, processor design; T_{clk} ← processor design, circuits, technology. Complex instructions: IC↓ CPI↑; short cycle: T↓ CPI↑ → balance.
- E.g. 3×10^9 instr, CPI 2, 3 GHz → 2 s. 50B instr: 10B branches×4 + 15B loads×2 + 5B stores×3 + 20B adds×1 = 105B cycles / 4 GHz = **26.25 s**.

> ⚠ **PITFALL** Amdahl's F must be a fraction of the **original execution time**. If you're told "20% of instructions are branches", convert using CPIs (or just compare Iron-Law times before/after). E.g. int 40%×1, br 20%×4→2, ld 30%×2, st 10%×3: CPI 2.1 → 1.7, speedup = **1.24** (not 1/(0.8+0.1) = 1.11).

- Make the common case fast: 20× on 10% → 1.105; 1.6× on 80% → 1.43; ∞ on 10% → caps at 1.111.
- E.g. (time fractions given): branches 20% of time, CPI 4→3 → 1/(0.8+0.2/1.333) = 1.05; clock 2→2.3 GHz → 1.15 (best); stores 10%, 3→2 → 1.034.
- **Diminishing returns**: after speeding up a part it becomes a smaller fraction (50/50, 2× → 1.33; another 2× → only 1.2) → re-evaluate the dominant part after every improvement.

3 · Pipelining

- 5 stages: **F**etch → **D**ecode/read regs → **A**LU → **M**EM → **W**rite reg. Latency unchanged, throughput × #stages.
- Laundry, 10 loads × 3 stages: unpipelined 30 h, pipelined 3+9 = **12 h**; 5 stages, 10 instr: 50 vs **14** cycles.
- Why CPI > 1: pipeline fill (negligible), **stalls** (data deps, bubbles), **flushes** (control deps, wrongly fetched instr → bubbles).
- Control deps: ~20% of instr are branches/jumps, >50% taken. Resolved in ALU (stage 3) → 2-cycle penalty each wrong fetch: CPI = 1 + 0.1×2 = 1.2.

Ex. 25% taken branches, 10-stage pipe, target known in stage 6: CPI = 1 + 0.25×5 = **2.25** (the "10 stages" is irrelevant).

Dependence vs Hazard

Type	Also called	Meaning
RAW	flow / **true**	later instr reads value written earlier
WAR	anti / false / name	later write must not precede earlier read
WAW	output / false / name	write order must be kept (final value)
RAR	—	**not** a dependence

- A dependence is a property of the **program** only; a hazard = a dependence that can cause incorrect execution **in a particular pipeline**.
- In the classic 5-stage pipe, WAR/WAW are **never hazards** (reads and writes in order); RAW is a hazard when there are **< 3** instructions between producer and consumer (read in D, write in W).
- 3-stage pipe (F/D | read+ALU | MEM+W): next instruction = hazard; one in between = no hazard.
- Handling: control hazard → **flush**; data hazard → **stall** or **forward** (prefer forwarding); load-use (value ready only at end of MEM, next instr needs it in ALU) → **must stall 1**, then forward.
- How many stages: more stages → bigger hazard penalties (CPI↑) but shorter cycle; performance-optimal ~30–40 stages; including power (latch energy, frequency) → modern **10–15 stages**.

4 · Branch Prediction

- At fetch you only know the **PC** (not yet decoded!); must guess next PC: is it a branch? taken? target? → boils down to "**is this a taken branch?**"
- **No prediction** (wait until sure): even non-branches cost 2 cycles (only after D do we know it isn't a branch), branches 3. **Predict not-taken** (PC+4): non-branch 1, branch 1 or 3 → never worse than refusing to predict.
- A branch mispredicted in the "shadow" of an earlier misprediction adds **no extra penalty** (it gets flushed itself).

Ex. Resolve in stage 3 (penalty 2) vs stage 10 (penalty 9), 20% branches: 50% accurate → 1.2 / 1.9; 90% → 1.04 / 1.18. No-predict vs perfect predictor on a 3-instruction loop: 7 vs 3 cycles (2.33×).
Ex. Not-taken accuracy ≈ 80% + 0.4×20% = 88%, mispred = 12%: 5-stage 1.24; 14-stage (penalty 10) 2.2; 4-wide 14-stage: 0.25 + 0.12×10 = 1.45. 99% predictor: 1.02 / 1.1 / 0.35 → deeper and wider pipes need better predictors.
Ex. P4 Prescott: CPI 0.5 with 1% mispred, penalty 30: ideal = 0.5 − 0.2×0.01×30 = 0.44; with 2% → 0.44 + 0.12 = **0.56**.

BTB / BHT

- **BTB** (Branch Target Buffer): indexed by PC → predicted next PC/target. Compared at execution; updated on mispredict. Must be single-cycle → small; only needs entries for instructions likely to execute soon.
- Index with the **lowest PC bits that actually vary** (not high bits: instructions of the same loop would collide). Fixed 4-byte aligned instr → **drop the low 2 bits**.
- **BHT**: indexed by PC, tiny entries (1-bit or 2BC) → can be large. Access the BTB only if BHT says taken; only taken branches go into the BTB.

▶ **HOW-TO** 1024-entry BTB, PC = 0x0000AB0C: drop low 2 bits → bits[11:2] = 0x2C3. 16-entry BHT, instructions from 0xC000: entry = (PC>>2) mod 16 (0, 1, 2…). BHT is accessed on every fetch; BTB only when predicted taken (with a perfect BHT = # of taken executions).

1-bit / 2-bit predictors

- 1-bit = last outcome. Problems: each anomaly causes **2 mispredictions** (the anomaly + the return to normal); short loops mispredict on exit and on re-entry; bad for patterns with similar T/N counts. 100-iteration loop example: 600+ executions, only 2 mispredictions.
- **2BC**: 00 strong N, 01 weak N, 10 weak T, 11 strong T; **MSB = prediction**, LSB = hysteresis/conviction. +1 on T, −1 on N, saturating. One anomaly costs only 1 mispredict; a behavior switch costs 2.
- Initial state: starting weak saves one mispredict when wrong, but risks 100% mispredict on alternating patterns; overall impact is tiny → 00 is easiest to initialize.
- **Every predictor has a 100%-mispredict sequence** (e.g. from 00: T, T, N, T, N…). 3-bit only helps when anomalies come in streaks → 2-bit is usually enough.

History-based

- Each entry: N-bit local history + 2^N 2BCs; history selects the counter. **N-bit history predicts patterns of length ≤ N+1**.
- 1-bit history + 2×2BC: NT alternation OK; NNT mispredicts once per round (100 rounds = **100** mispredictions). 2-bit history: NNT uses 3 of 4 counters (11 never occurs); NT uses 2.
- Waste: only ~N+1 counters used. An 8-iteration loop needs 8-bit history → 256 counters, only 9 used.

N	bits/entry	×1024 entries	NNNT?	2BCs used for NT
1	1+4=5	5,120	✗	2
4	4+32=36	36,864	✓	2 / 16
8	8+512=520	532,480	✓	2 / 256
16	16+131072	134,234,112	✓	2 / 65536

Shared counters: PShare / GShare

- **PShare**: low PC bits index the PHT (**private** per-branch history, history bits only) → history XOR PC → index a shared table of 2BCs (2 bits each). Conflicts possible but rare with a big table. Cost example: 2^{11}×11 + 2^{11}×2 ≈ **26 Kbit**. An always-taken branch uses just 1 counter.
- **GShare**: one **global** history (every branch outcome shifted in) XOR PC. Good for **correlated branches** (if (shape==square) … if (shape!=square)).
- PShare is good for self-patterns (even/odd, short loops); GShare needs longer history for the same job. E.g. even/odd branch inside a for loop: PShare history 1 bit; GShare needs **3 bits** (two other branches in between).
- **Tournament**: two good predictors + meta-predictor (a 2BC per PC choosing which to use); both are always trained; meta counts only when **one is right and the other wrong** (GShare right → down, PShare right → up).
- **Hierarchical**: cheap predictor + expensive good one; a branch is inserted into/updated in the good one only when the cheap one fails (tagged) → good one can have few, very strong entries. Usually beats tournament. Pentium M: 2BC → local (use if tag hit) → global (preferred if tag hit); 2BC wrong → insert into local; local wrong → insert into global.
- Combination answer template: **Hierarchical(2BC , Tournament(PShare, GShare))**.

RAS (Return Address Stack)

- Function returns: direction easy (always taken), target hard (many call sites) → BTB keeps mispredicting. RAS: push return address on call, pop on return.
- RAS full: **wrap around** (overwrite oldest) beats don't-push — many small function calls, sacrifice the final return of large functions. It's only a predictor; wrong = just a mispredict.
- How do we know it's a RET at fetch? A simple 1-bit predictor, or **predecoding** (extra bits per instruction in the I-cache: is RET / is branch / length).

5 · Predication

- Predict right → lose 0; wrong → throw away ~50 instr (12 stages × 4-wide ≈ 48). Predication: do both paths, always waste half.
- **Loops**: predict (predication diverges exponentially); **call/return**: predict; **large if-then-else** (100 instr each): predict (waste 100 > mispredict cost 50); **small if-then-else** (5 each): waste 5 vs 0.1×50 = 5 → predicate if accuracy < 90%.
- **If-conversion**: compiler computes both paths then selects. Selecting with a branch = pointless. Needs conditional-move instructions.
- **MIPS**: `MOVZ Rd,Rs,Rt`: if Rt==0 then Rd←Rs; `MOVN`: if Rt≠0. x86: `CMOVcc` (uses flags).

Ex. `BEQZ R1,Else; ADDI R2,R2,1; B End; Else: ADDI R3,R3,1; End:` → `ADDI R4,R2,1; ADDI R5,R3,1; MOVN R2,R4,R1; MOVZ R3,R5,R1` (4 instr, no branches).

▶ **HOW-TO** Break-even: if-converted 4 instr vs original avg 2.5 instr + p×30 → p = 1.5/30 = 5% → if-convert when accuracy is **< 95%**. (Other example: 80% accurate, penalty 40: 2.5 + 0.2×40 = 10.5 ≫ 4.)

- Downsides of the CMOV approach: needs compiler support (old code doesn't benefit), **more registers**, **extra select instructions**.
- **Full predication** (Itanium): every instruction has a qualifying predicate; 41-bit instructions, low 6 bits pick one of 64 1-bit predicate registers. `cmp.eq p1,p2 = R1,0` sets complementary predicates, then `(p2) ADDI R2,R2,1`, `(p1) ADDI R3,R3,1` → no extra registers or moves.

Ex. Full-pred 3 instr at CPI 0.5 = 1.5 cycles; branchy avg 2.5 instr = 1.25 cycles + p×10 → p = 2.5% → if-convert when accuracy < **97.5%**.

6 · ILP

- Can't execute everything in the same cycle: RAW deps require values to be produced first (forwarding can't "time travel").
- WAW example: I2 delayed by RAW, I5 writes R4 first → wrong final value → write in order or **rename**.
- False/name deps (WAR, WAW) exist only because a register name is reused → removed by **register renaming** (alternative: keep all versions = too complex).
- **RAT** (Register Allocation/Alias Table): architectural register → current physical register.

▶ **HOW-TO** Renaming, per instruction: ① look up **source** registers in the RAT → physical names; ② **then** allocate a new physical register for the **destination** and update the RAT. (Reversed order = instruction waits for its own result!)

ILP definition: processor that does a whole instruction in 1 cycle, unlimited width, perfect same-cycle branch prediction, no structural limits, renaming → IPC limited only by **RAW**. **ILP is a property of the program**, not of any processor ("ILP on an i7" is meaningless).

▶ **HOW-TO** Computing ILP: no need to actually rename — **draw only RAW edges**, ignore WAR/WAW and control deps; cycle(instr) = 1 + max(cycle of its producers); `ILP = #instructions / last cycle` (don't invert). A branch still waits for its own RAW, but instructions after it don't wait for it.

- IPC is also limited by issue width, in-order execution, number of functional units (e.g. only 1 ALU). E.g. ILP 2.5; 2-issue OoO with 1 MUL + 2 ALUs → 2.5; with only 1 ALU → 1.25. 3-issue in-order, ILP 3 → IPC 2 (in-order stops at the first dependence).
- Narrow in-order: limited by issue width; wide in-order: limited by in-order → need **wide + OoO + renaming**.

7 · Instruction Scheduling (Tomasulo)

- IBM 360 (40+ years old); back then FP-only, small window, weak exception handling; modern OoO is essentially the same design.
- Structures: **IQ** (in order) → per-unit **RS** (reservation stations); **RAT**; **RF**; results broadcast on the **CDB** (common data bus) to the RF and all RSs; loads/stores: address generation then load/store buffers, memory accessed **in order** (original Tomasulo did not reorder L/S).
- Three steps: **Issue** (IQ → RS, in order) → **Dispatch** (RS → execution unit, out of order) → **Write result / Broadcast** (out of order).

Issue

- Take the next IQ instruction (**must be program order**, otherwise renaming breaks); need a free RS of the **right type**, else nothing issues this cycle (later ones can't issue either).
- Sources: RAT empty → read value from RF; RAT has a tag → record which RS to wait for. Then RAT[dest] ← this RS name.

Dispatch

- An instruction with all operands ready may dispatch; one per unit per cycle. If several are ready: **oldest first** (common), most-dependents-first (hard to implement), random (still correct).
- Valid reasons a ready instruction didn't dispatch: it was issued last cycle; the unit was taken by another. It does **not** wait for older instructions to dispatch (that would be in-order).

Write Result

- Put (tag, value) on the CDB → every RS compares both operand tags and captures → if RAT[reg]==tag: write RF and clear the RAT entry (point back to RF) → free the RS.
- Simultaneous completions with 1 CDB: usually give priority to the **slower unit** (mul/div; more instructions waiting on it). More CDBs → comparators per RS × #buses.

⚠ **PITFALL Stale broadcast**: the tag is no longer in the RAT (dest was renamed by a later instruction) → RSs still capture, but **do NOT write the RF or change the RAT**. Don't clear that RAT entry out of habit!

Same-cycle rules (defaults; the problem may say otherwise): issue → dispatch in the same cycle: usually **no**; capture → dispatch same cycle: usually no; issue and broadcast writing the same RAT entry in the same cycle: **the issuing instruction wins** (it is newer).
Tomasulo: issue **in order**; dispatch and write result **out of order**.

▶ **HOW-TO** Timing table:
① Issue(i) = Issue(i−1)+1, but if no free RS → wait until the cycle **after** some RS frees (an RS freed in a cycle can't be reused that same cycle; in Tomasulo an RS is freed at **broadcast**).
② Exec start = max(Issue+1, last operand's broadcast cycle + 1) (unless capture & dispatch in the same cycle is allowed — check rules).
③ Write = exec start + latency (long example: 2-cycle exec → writes in the 3rd cycle); delay on CDB conflicts by priority.
④ Unpipelined unit: next op waits until the previous finishes. Annotate each delay with its reason ("waiting for RS", "waiting for R2").

8 · ReOrder Buffer (ROB)

- Tomasulo problems: ① **imprecise exceptions** (DIV by zero detected at cycle 40, later instructions already wrote registers); ② **branch misprediction recovery** (wrong path already wrote R3); ③ **phantom exceptions** (an instruction that should never have executed raises an exception).
- Fix: execute out of order, broadcast out of order, but **write registers in order (commit)**.
- ROB entry: **value**, **done** bit, **dest reg**; kept in program order; issue pointer (where new instr go) and commit pointer (oldest).

Step	Tomasulo	+ROB
Issue	needs free RS; RAT→RS#	needs free RS **and** next ROB entry; RAT→**ROB#**
Dispatch	RS kept	**free the RS** at dispatch (the ROB entry is now the name)
Broadcast	write RF, update RAT, free RS	tag = ROB#; RSs capture; write ROB value, done=1; **RF/RAT untouched**
Commit	—	in order: commit head only if done; value written to RF **unconditionally**; RAT pointed back to RF only if RAT[dest]==this ROB#; free ROB entry

- ROB's roles: remember program order; hold results until commit; serve as the result's name/tag. **Not**: storing source operands (RS's job), choosing the unit (RS's job; the ROB is unified).
- With the same # of RSs, issue stalls for lack of an RS are more likely **without** a ROB (RS held until broadcast).
- **Mispredict recovery**: when the branch executes, mark it mispredicted in the ROB; when it **commits**, the RF holds exactly the state at the branch → empty the ROB (issue ptr = commit ptr), point the whole RAT to the RF, flush RSs and units, fetch from the correct PC.
- **Exceptions**: record the exception as the "result" in the ROB; at commit: everything before is committed, it and everything after is not → flush, jump to handler (precise). Phantom exceptions never reach commit.
- The programmer only sees commit ("officially executed").
- Why commit always writes the RF (even if soon overwritten): so the RF is always the exact committed state and we can flush at any moment.

▶ **HOW-TO** ROB timing: Issue (in order + RS + ROB entry) → Exec = max(Issue+1, operands ready) → Write = Exec+lat → **Commit = max(Write+1, previous Commit+1)** (or per commit width). **Check whether the problem frees RSs at dispatch or at broadcast.** E.g. a 40-cycle DIV first → every later instruction commits after cycle 43 (44, 45, …).

- **Unified RS**: one shared pool, better utilization (one full class doesn't block issue); cost: more complex dispatch selection.
- **Superscalar**: fetch, decode, issue (several consecutive, in order; if one can't, later ones can't), dispatch, broadcast (comparators per RS $\propto$ #buses), commit (only after all earlier commit) must all be >1/cycle; **the narrowest stage limits IPC**.
- **Terminology**: Issue = allocate = (some say) dispatch; Dispatch = execute = (some say) issue; Commit = retire = complete = graduate.
- In an OoO processor, **in order**: fetch, decode, issue, commit (and after); **out of order**: dispatch, execute, broadcast.

9 · Memory Ordering (LSQ)

- Register deps are handled by renaming + Tomasulo; loads/stores can depend through **memory addresses** (RAW: SW→LW same addr; WAR: LW→SW; WAW: SW→SW).
- Stores write cache/memory only at **commit** (they may be cancelled before that; undoing a memory write is very hard).
- **LSQ**: like the ROB (in order, removed at commit) but only loads/stores; fields: L/S bit, address, value, done.
- When a load has its address: search older stores for the **most recent** matching address → match: **store-to-load forwarding** (no memory access); no match: access the cache.

Older store address still unknown:
① execute all L/S in order (slowest, correct); ② wait until all older store **addresses** are known (address match but value not ready → wait for value); ③ **go anyway** (modern processors): no match → go to memory; later when a store resolves its address, check younger loads that already executed to the same address → **replay that load and everything after it**. Predictors can pick which loads should wait.

Ex. 3 LW/SW pairs, every load misses (40 cycles): out of order 44 cycles; in order: LW 1→41, SW 42, LW 43→83, SW 84, LW 85→125, SW **126**.

- Issuing a L/S needs a **ROB entry + LSQ entry** (LSQ acts as the RS). Execute = compute address + (load) get the value.
- A load **broadcasts** its value; a store does **not broadcast and has no ROB result**. A load forwarded from a store gets its value from the **LSQ entry** (not CDB/RS/ROB).
- Commit: load = free ROB + LSQ entries; store also sends its write to the cache. Cache/memory always = committed state.

10 · Compiler ILP

- The compiler can: ① raise the program's own ILP (break dependence chains); ② bring independent instructions closer (hardware window/ROB is limited).
- **Tree height reduction**: `((R2+R3)+R4)+R5` → `(R2+R3)+(R4+R5)`; relies on **associativity**, not valid for all ops (careful with FP).

Ex. `R1+R2–R3+R4–R5+R6–R7` (6 serial instr, ILP 1) → `R10=R1+R2; R11=R4+R6; R10=R10+R11; R11=R3+R5; R11=R11+R7; R10=R10–R11` → 3 cycles, ILP **2**.

- **Instruction scheduling**: move independent instructions into stall slots. Moving `ADDI R1,R1,4` up past a store → the store becomes `SW –4(R1)`. Moving a load earlier may require a **different register** (R1→R10) to avoid clobbering.
- E.g. LW 2 cycles, ADD/SW 1: `LW,ADD,SW,LW,ADD,SW` 8 cycles → 6 after scheduling + renaming.
- After if-conversion the code is branch-free → can schedule across the former then/else boundaries. Loops can't be if-converted (a new predicate every iteration) → use **loop unrolling**.

⚠ **PITFALL Unroll once = each new iteration does 2 original iterations**; unroll twice = 3. Copy the body, adjust the second copy's offsets (e.g. `–4(R1)`), step the pointer by 2× (−8), keep a single branch.

- Unrolling benefits: ① **IC**↓ (loop overhead removed: 5×1000 = 5000 → 8×500 = 4000); ② more room to schedule → **CPI**↓. 4-issue in-order example: original CPI 3/5 → scheduled 2/5; unrolled once unscheduled 5/8, scheduled **3/8**.
- Downsides: **code bloat**; unknown trip count / not a multiple (beyond this course).
- **Function inlining**: removes call/return and parameter-setup overhead (IC↓) and lets the body be scheduled with the caller (CPI↓). Downside: code bloat → inline only **small functions**. E.g. 10 cycles → 7 after inlining.
- Others: **software pipelining** (treat the loop as a pipeline, overlap different stages of different iterations, no code bloat); **trace scheduling** ("if-conversion on steroids": schedule the common path as one block; leaving the trace jumps to compensation code).

11 · VLIW

	OoO superscalar	In-order superscalar	VLIW
Per cycle	≤N instr	≤N instr	1 large instr (= N ops)
Finding indep. instr	search a window ≫ N	only the next N	doesn't; compiler does
HW cost	high	lower	lowest
Compiler	helps	needs it	**completely depends**

- Code size: 4000 B program of 32-bit instr → 128-bit/4-op VLIW: **4000 – 16000 B** (all-dependent code: 1 op + 3 NOPs each).
- Good: compiler has plenty of time to schedule; simpler hardware; energy efficient; great on **loops and regular code** (arrays, matrices).
- Bad: latencies not fixed (cache misses); irregular code (decisions, pointers: AI, linked lists); **code bloat** (NOPs).
- ISA features: normal opcodes, **full predication**, **many registers** (needed for scheduling), **branch hints**, **instruction compaction** (stop bits to cut NOPs).
- Backward compatibility: fetching 2 old 64-bit VLIW instructions per cycle → **not really VLIW** (they may depend on each other → needs HW dependence checks).
- Examples: Itanium (hardware ended up very complex, still bad on irregular code); **DSPs** (regular FP loops, excellent performance and energy). Best target: "adding up many numbers"; poor: maze path finding, counting linked-list elements.

12 · Cache Review

- **Temporal locality**: same address accessed again soon (loop variable j, sum); **Spatial**: nearby addresses soon (array elements arr[i]; arr itself has no temporal locality). New Year's Eve is a counter-example.
- Cache = small, fast, inside the processor (borrow the book and bring it home, rather than going to the library each time / building a library at home).
- AMAT = HT + MR×MP; alternative form `(1–MR)×HT + MR×(HT+MP)`. Balance hit time (small/fast) vs miss rate (large/smart). Good cache: HT ≪ miss time, miss time ≈ MP, hit rate ≈ 1.
- L1: **16–64 KB**, ~90% hit, **1–3 cycles**. Block **32–128 B**: ≥ largest single access, exploits spatial locality; too big (1 KB) wastes capacity. Must be a **power of 2** (48 B is bad) and **aligned** (blocks start at multiples of the block size).
- Line = slot in the cache; block = memory content placed in it (same size).

Ex. 32 KB, 64 B blocks, N variables with no spatial locality: each occupies a block → N ≤ 32K/64 = **512**.

- **Tag**: never includes offset bits; contains at least 1 block-number bit; in DM/SA excludes index bits (all blocks in a set share them).
- **Valid bit**: all set to 0 at power-up; hit = tag match **and** valid=1 (tag/data initial values don't matter).
- Types: **direct-mapped** (1-way; fast, cheap, energy-efficient, but **conflicts**), **N-way set-associative** ("2-way" = 2 blocks per set, not 2 sets), **fully associative** (no index bits).

▶ **HOW-TO** Address breakdown: **offset first** ($\log_2$ block) → then index ($\log_2$ #sets, #sets = size/block/ways) → rest is tag. Conflict = same index, different block number (same block is not a conflict).
E.g. 16 KB DM, 256 B blocks: offset 8 bits (low 2 hex digits), 64 sets → index 6 bits; 0x12345678 conflicts with 0x11115678 and 0x12341666, same block as 0x12345677.
E.g. 256 B, 32 B blocks, 2-way, 32-bit addr: offset [4:0], index [6:5], tag [31:7].
E.g. 4 KB 4-way 64 B, 64-bit: offset 6, 16 sets → index 4, tag 54; per-line overhead valid1 + dirty1 + tag54 + LRU2 = 58 bits.

Replacement

- Random, FIFO, **LRU** (good but costly), **NMRU** (track MRU, pick randomly among the rest).

▶ **HOW-TO** LRU counters (N-way, values 0…N−1 all distinct, 0 = LRU): access block X (old value c) → X set to N−1; every counter **> c** decrements, those < c unchanged. Replace the block with counter 0 (new block then becomes N−1). Re-accessing the MRU block: no change. Check all counters are distinct after each step.

Write policy

- Allocate: **write-allocate** (bring block in on a write miss; common) vs no-write-allocate.
- **Write-through** (every write also goes to memory) vs **write-back** (write memory only on replacement; needs a **dirty bit**; clean blocks aren't written back). Write-back usually pairs with write-allocate.
- A write hit just sets dirty = 1 (no need to check first). If valid = 0, dirty/tag are meaningless.

Ex. One DM line, start V=0, D=1, tag=A; R A, R B, W B, R C, R D, W D → 4 misses, 1 write-back (B), final V=1, D=1, tag=D.

13 · Virtual Memory

- Program's view: huge linear address space (code, static, heap↑ … stack↓), every process starts at 0; hardware's view: the actual DRAM modules. 16 processes with 4 GB spaces can run on 4 GB / 16 GB / … of physical memory.
- Physical address ↔ physical location is one-to-one. Most virtual memory (between heap and stack) is never touched.
- **Page** (virtual, aligned, typically 4 KB) ↔ **frame** (physical slot); OS keeps a **page table** per process; shared memory = pages of two processes mapped to the same frame. Pages that don't fit are on **disk** and must be brought in first.
- Translation: VA = VPN | page offset; PT[VPN] → PFN; PA = PFN | offset (offset unchanged).

Ex. 2 GB physical, 4 GB virtual, 4 KB pages: frames = $2^{31}/2^{12} = 2^{19}$; PT entries per process = $2^{32}/2^{12} = 2^{20}$.
Ex. 16-bit VA, 4-entry PT → VPN = top 2 bits, offset = 14 bits; 20-bit PA = PFN (6 bits) | 14-bit offset.

- A flat PTE ≈ physical address size (PFN + protection/present bits). 4 GB/4 KB × 4 B = **4 MB per process** (even if it uses very little). Impossible for 64-bit spaces.

Ex. 8 B PTE, 4 KB pages, 2 processes, 32-bit VA (regardless of using 1 MB / 1 GB, 2 GB physical): $2 \times 2^{20} \times 8$ = **16 MB**.

Multi-level page tables

- VPN split into outer | inner; outer entries point to inner tables; an unused region → that inner table **doesn't exist**. All inner tables together = flat size (+ outer); savings come from omitted inner tables.

Ex. 32-bit, 4 KB, 10|10|12, 8 B PTE, using only the lowest ~16 pages (VPN 0x00000–0x00010) and the top 16 pages (VPN 0xFFFF0–0xFFFFF): flat = $2^{20}\times8$ = 8 MB; two-level = outer 8 KB + 2 inner × 8 KB = **24 KB**.

Ex. 64-bit, 64 KB pages, 8 B PTE, using 0–4 GB: flat = $2^{48}\times8 = 2^{51}$ B; 4-level (48 = 4×12): innermost tables needed = 2^{16} pages / 2^{12} = 16, plus 1 at each of the 3 upper levels = 19 tables × ($2^{12}\times8$ = 32 KB) = **608 KB**.

- Page size: larger → smaller page tables; but more **internal fragmentation**. Sweet spot: a few KB to a few MB; x86 uses 4 KB (memory waste was a bigger worry back then).
- Cost: PT lives in memory → every load/store first does n levels of PT accesses.

Ex. 1 cycle for address, cache hit 1, memory 10, 90% hit, 3-level PT: PTEs not cacheable → 1 + 3×10 + 1 + 0.1×10 = **33**; PTEs cacheable → 1 + 3×(1+0.1×10) + (1+0.1×10) = **9** (6 cycles spent on translation).

TLB

- Caches only the **final** translation (multi-level → still 1 access); one entry covers a whole page → tiny and fast. 16 KB of data needs only 4 entries.
- Miss handling: **software** (OS fills the TLB; PT can be any structure: tree, hash table; embedded) vs **hardware** page walker (PT must be flat/multi-level format; faster; x86).
- Organization: **fully / highly associative**; 64–512 entries; not enough → two-level TLB (L1 single-cycle; L2 thousands of entries, several cycles).
- Entries needed: 32 KB cache / 64 B blocks, 4 KB pages → between 8 (cache/page) and 512 (#blocks) → answer "8 to 512".

Ex. Sweep a 1 MB array byte by byte 10 times, 4 KB pages → 256 pages; L1 TLB 128 entries, L2 1024 (DM, initially empty): L1 misses = 10×256, hits = 10×4095×256; L2 misses = 256, hits = 9×256.

14 · Advanced Caches

Three families: reduce **hit time** / reduce **miss rate** / reduce **miss penalty**.

Reduce Hit Time

- Smaller / less associative: HT↓ but MR↑ — balance.
- **Pipelined cache**: stage 1 read tags + valid; stage 2 determine hit, start data read; stage 3 finish data read. L1 caches with 2–3 cycle hits are usually pipelined (hit time includes waiting for the previous access).
- **PIPT**: TLB then cache, hit latency = TLB + cache.
- **VIVT**: access with the VA directly; no translation on hits. Problems: still need TLB **permission bits**; **flush on context switch**; **aliasing** (mmap two VAs → same PA, may land in different sets; write A, read B doesn't see it).
- **VIPT**: VA index bits select the set while the TLB is accessed in parallel; compare tag with the PA. HT = max(cache, TLB) ≈ cache; **no flush**; if all index bits come from the page offset → **no aliasing**.

Max alias-free VIPT size = associativity × page size. E.g. 4-way, 16 B blocks, 8 KB pages: offset 4 + index 9 = 13 → $2^9\times16\times4$ = **32 KB**. 4 KB pages, 32 B blocks → index ≤ 7 bits (128 sets). Intel: P4 4-way 16 KB; Core 2 – Haswell 8-way 32 KB; Skylake (rumored) 16-way 64 KB. Growing it requires more associativity.

- **Way prediction**: guess one way first (like a small DM cache), on a wrong guess check the whole set → miss rate of SA, hit time mostly like DM. Not applicable to direct-mapped.

Ex. 32 KB 8-way: 90%, HT 2, MP 20 → AMAT 4. 4 KB DM: 70%, HT 1 → 1 + 0.3×20 = 7. Way prediction: 0.7×1 + 0.3×2 = 1.3, + 0.1×20 → **3.3**.

- Replacement vs HT: Random updates nothing on hits (fast, worse MR); **LRU** may update N counters on every hit (and must compare them even if not) → slower, more power.
- **NMRU**: one MRU pointer per set ($\log_2$N bits, N× less state than LRU); protects only the MRU, no order among the rest.
- **PLRU** (pseudo-LRU): 1 bit per line; set on access; replace a line with bit 0; when the last 0 would become 1, clear all **other** bits. Between NMRU (one 1) and true LRU (one 0 left); hits only set a bit.

Ex. 4-line fully-assoc NMRU: A A B A C A D A E A A A A B → first 4 distinct blocks fill empty lines; E replaces one of B/C/D → the final B may hit or miss → min **5**, max **6** misses.

Reduce Miss Rate

3Cs: **Compulsory** (first access; even an infinite cache has them); **Capacity** (would miss even in a **fully associative** cache of the same size); **Conflict** (would NOT miss in a fully associative cache of the same size — limited associativity). Bigger cache → capacity↓; more associativity / better replacement → conflict↓ (both hurt HT).

- **Larger blocks**: MR↓ with good spatial locality, MR↑ with poor locality (junk occupies capacity). Small caches turn up early (4 KB ~64 B), large caches later (256 KB ~256 B). With good locality: reduces compulsory (almost always), capacity (yes), conflict (maybe).

- **Prefetching**: good guess → miss eliminated; bad guess → **cache pollution** (may even create new misses).
- Prefetch instructions: `prefetch(a[i+pdist])`; too small → too late (still wait, but less); too large → too early (evicted before use); and the right value changes across hardware generations.

Ex. a[i] += b[i][j], 8 B elements, 16 KB fully-assoc LRU, inner iteration 10 cycles, MP 200: b's pdist = 200/10 = **20** (inner loop); prefetch a at **i+1** (one outer iteration = 10000 cycles > 200; each outer iteration brings only 8000 B < 16 KB so it stays; i+20 would be evicted = premature).

- Hardware prefetchers: **stream buffer** (next few sequential blocks), **stride** (fixed distance d), **correlating** (remembers "after A comes B" → good for repeated linked-list traversals).
- **Loop interchange**: C arrays are row-major → inner loop should walk the last index (`a[i][j]`, j inner); compiler must prove no dependences between iterations before swapping.

Reduce Miss Penalty

- **Blocking** cache: one miss at a time. **Non-blocking**: **hit-under-miss**, **miss-under-miss** → overlapping misses = **MLP** (memory-level parallelism).
- **MSHRs**: on a miss, check MSHRs: no match → true miss, allocate a new MSHR, record the instruction to wake up, send request; match (block already in flight) → **half-miss**, just add the instruction to that MSHR, no new request. Data returns → wake all, free the MSHR. Even 2 help a lot; 16–32 even better.
- No benefit: programs that always hit; a miss every 1000 instructions (ROB fills before the next miss is seen). Miss every 10 or every 2 instructions → benefits.
- **Multi-level caches**: L1 miss → L2; last level = **LLC** (misses go to memory). L1 capacity < L2, L1 latency < L2; L2 gets **fewer** accesses than L1; associativity need not match (L1 usually lower).

Ex. 16 KB: HT 2, 90% → 2 + 0.1×100 = 12; 128 KB alone: HT 10, 97.5% → 12.5; no cache: 100; L1+L2: L2 local hit 75% → 2 + 0.1×(10 + 0.25×100) = **5.5**.

- **Local** miss rate = misses / accesses to this cache; **global** = misses / all processor accesses; **MPKI** = misses per 1000 instructions. L1 90%: local = global = 10%; L2 hits half of L1 misses: local 50%, global **5%**. L2's local hit rate looks low because L1 filters out the easy accesses.
- **Inclusion**: by default **neither inclusive nor exclusive** (accesses to L1-hot blocks never reach L2, so L2's LRU evicts them). To enforce inclusion: add an inclusion bit in L2 and never replace blocks still in L1. With inclusion, an L1 dirty write-back is **always an L2 hit**; without it, may be hit or miss.

> **Exam tip**: for Tomasulo/ROB/cache simulation problems, first copy down the given rules (latencies, when RSs free, whether same-cycle issue/capture/dispatch is allowed, # of CDBs, pipelined units); annotate every delay with its reason for partial credit.